

Furqan RaaS (Reasoning-as-a-Skill) — Product Documentation

Complete Technical Reference

Project: Furqan RaaS — MCP-based reasoning skill for AI agents

Version: 0.1.0

Date: March 21, 2026

Location: furqan-raas/

Protocol: JSON-RPC 2.0 over stdio (MCP-compatible)

Table of Contents

- [1. What is RaaS?](#)
- [2. Architecture](#)
- [3. Package Structure](#)
- [4. MCP Server](#)
- [5. All 5 Tools — API Reference](#)
- [6. Intent Detection & Safety](#)
- [7. Installation Guide](#)
- [8. SKILL.md Reference](#)
- [9. Example Calls & Responses](#)
- [10. Test Coverage](#)

1. What is RaaS?

Reasoning-as-a-Skill (RaaS) packages the AI-Furqan reasoning engine as an MCP skill that any AI agent can call. Instead of needing a web API, agents simply start the RaaS process and communicate via JSON-RPC over stdio.

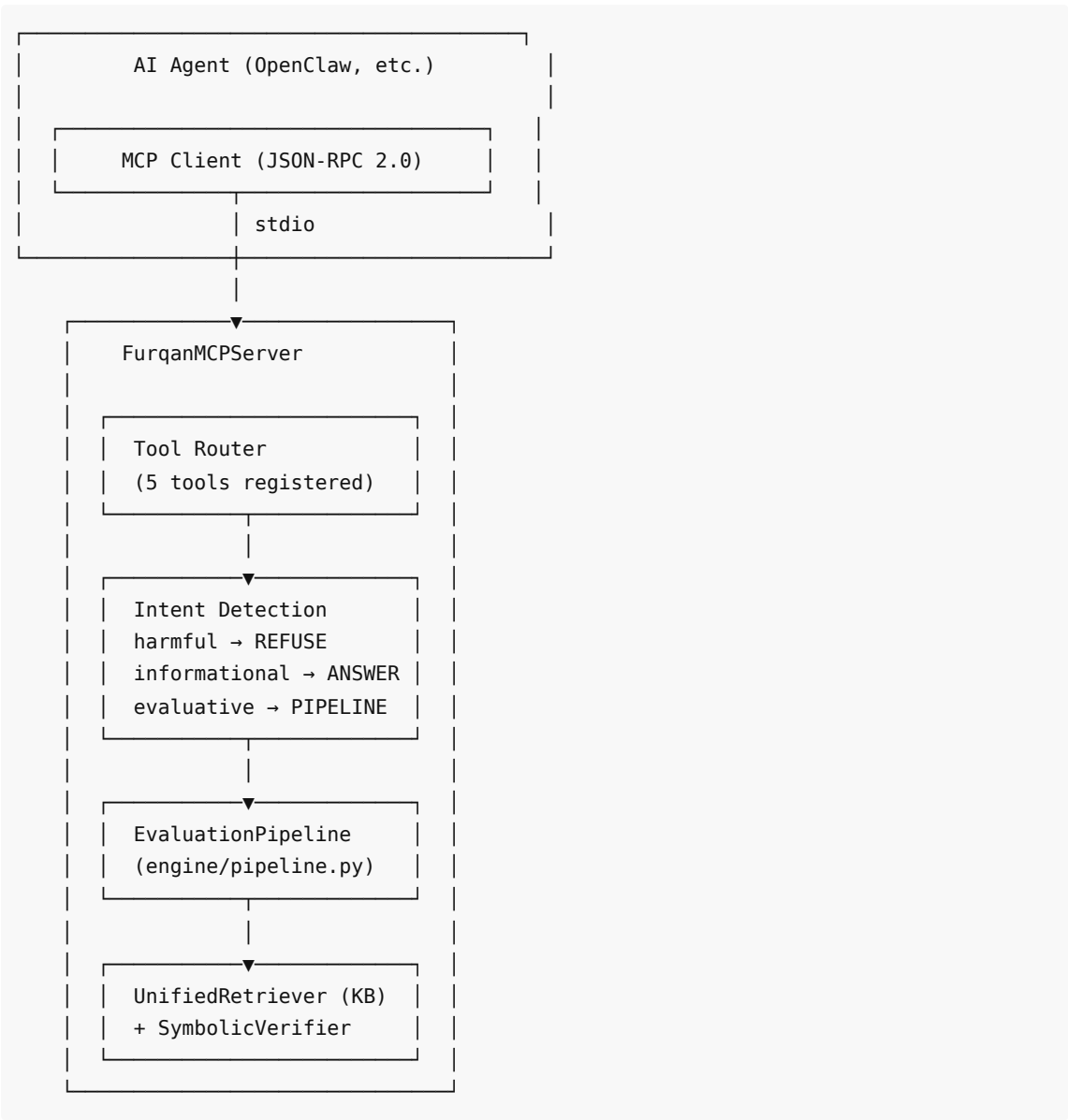
Why RaaS?

- **Agent-native:** Works with OpenClaw, Claude Code, Cursor, and any MCP-compatible client
- **No API server needed:** Runs as a subprocess — start, use, done
- **Full pipeline access:** Same 4-gate evaluation + Z3 verification as the main engine
- **Knowledge-grounded:** Optionally retrieves from Quran, Hadith, and Fiqh collections
- **Safety built-in:** Harmful queries refused, informational queries shortcuted

What's the Difference from the REST API?

Feature	REST API	RaaS (MCP Skill)
Transport	HTTP	stdio (JSON-RPC 2.0)
Auth	API keys	Process-level isolation
Client	Any HTTP client	MCP-compatible agents
Startup	Server must be running	On-demand subprocess
Use case	Web apps, dashboards	AI agent tools

2. Architecture



3. Package Structure

```
furqan-raas/
├── pyproject.toml           # Package metadata + dependencies
├── README.md               # Quick start guide
├── SKILL.md                # MCP skill manifest
├── src/
│   ├── furqan_raas/
│   │   ├── __init__.py
│   │   ├── __main__.py    # Entry point: python -m furqan_raas
│   │   └── mcp_server.py  # MCP server implementation
└── tests/
```

```
|— __init__.py
|— test_mcp_server.py          # 31 tests
```

4. MCP Server

Server Identity

```
SERVER_NAME = "furqan-reasoning"
SERVER_VERSION = "0.1.0"
PROTOCOL_VERSION = "2024-11-05"
```

Initialization

```
class FurqanMCPServer:
    def __init__(
        self,
        llm_fn: Optional[Callable[[str], str]] = None, # LLM for evaluation
        retriever: Optional[UnifiedRetriever] = None,   # KB search
        verifier: Optional[SymbolicVerifier] = None,    # Z3 verification
    ):
```

All three dependencies are optional:

- Without `llm_fn` : Cannot evaluate or explain, but can retrieve and list domains
- Without `retriever` : No knowledge base search
- Without `verifier` : No Z3 verification (skipped in "quick" depth)

JSON-RPC Protocol

The server reads newline-delimited JSON-RPC 2.0 from stdin and writes responses to stdout:

```
→ {"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {}}
← {"jsonrpc": "2.0", "id": 1, "result": {"protocolVersion": "2024-11-05", "capabilities":
{"tools": {"listChanged": false}}, "serverInfo": {"name": "furqan-
reasoning", "version": "0.1.0"}}}

→ {"jsonrpc": "2.0", "id": 2, "method": "tools/list", "params": {}}
← {"jsonrpc": "2.0", "id": 2, "result": {"tools": [...]}}

→ {"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params":
{"name": "furqan_evaluate", "arguments": {"question": "Is riba permissible?"}}}
← {"jsonrpc": "2.0", "id": 3, "result": {"content": [{"type": "text", "text": "..."}]}}
```

5. All 5 Tools — API Reference

Tool 1: `furqan_evaluate`

Full axiom-anchored evaluation with 4-gate scoring and optional Z3 formal proof.

Input Schema:

Parameter	Type	Required	Default	Description
question	string	✓	—	The question or claim to evaluate
domain	string	✗	"islamic"	Knowledge domain
depth	string	✗	"standard"	quick (no Z3), standard, deep (extra correction)

Output (evaluative):

```
{
  "type": "evaluation",
  "response": "Final judgment text...",
  "verdict": {
    "gate_scores": [
      { "name": "Source-Integrity", "score": 40, "result": "Fail", "reasoning": "..."},
      { "name": "Structural-Consistency", "score": 30, "result": "Fail", "reasoning": "..."},
      { "name": "Mediation-Zeroing", "score": 20, "result": "Fail", "reasoning": "..."},
      { "name": "Origin-Aware", "score": 0, "result": "Fail", "reasoning": "..."}
    ],
    "z3_verification": {
      "consistent": false,
      "proof": "The predicates contradict the Al-Furqan axiom system.",
      "contradictions": [...],
      "verification_time_ms": 45.2
    },
    "total_score": 22,
    "origin_gate": "Fail"
  },
  "sources": [
    { "source": "quran", "reference": "Quran 2:275", "content_en": "..."}
  ],
  "evaluation_id": "eval_abc123",
  "processing_time_ms": 4523.7
}
```

Output (informational — shortcut):

```
{
  "type": "informational",
  "response": "Direct factual answer...",
  "category": "religion",
  "sources_suggested": [...],
  "related_topics": [...],
  "evaluation_id": "eval_def456",
}
```

```
"processing_time_ms": 1200.5
}
```

Output (harmful — refused):

```
{
  "type": "evaluation",
  "refused": true,
  "reason": "This question has been flagged as potentially harmful and cannot be evaluated.",
  "evaluation_id": "eval_ghi789"
}
```

Tool 2: furqan_verify

Quick claim verification against the knowledge base. Returns confidence score and citations.

Input Schema:

Parameter	Type	Required	Default	Description
claim	string	✓	—	The claim to verify
domain	string	✗	"islamic"	Knowledge domain

Output:

```
{
  "type": "verification",
  "claim": "Riba is prohibited in Islam",
  "confidence": 0.8,
  "sources_found": 4,
  "citations": [
    {
      "source": "quran",
      "reference": "Quran 2:275",
      "content_ar": "...الَّذِينَ يَأْكُلُونَ الرِّبَا",
      "content_en": "Those who consume interest..."
    }
  ],
  "processing_time_ms": 120.3
}
```

Confidence calculation: $\min(1.0, \text{sources_found} / 5.0)$

Tool 3: furqan_retrieve

Pure knowledge retrieval — search verified knowledge bases and return formatted sources.

Input Schema:

Parameter	Type	Required	Default	Description
query	string	✓	—	Search query (Arabic or English)
sources	string[]	✗	["quran", "hadith", "fiqh"]	Which collections
limit	integer	✗	5	Max results per source

Output:

```
{
  "type": "retrieval",
  "query": "الربا",
  "results": [
    {
      "source": "quran",
      "reference": "Quran 2:275",
      "content_ar": "...",
      "content_en": "...",
      "metadata": {"surah": 2, "ayah": 275, "juz": 3}
    }
  ],
  "total_found": 9
}
```

Tool 4: furqan_explain

Get a sourced explanation of a topic grounded in verified knowledge.

Input Schema:

Parameter	Type	Required	Default	Description
topic	string	✓	—	Topic to explain
domain	string	✗	"islamic"	Knowledge domain

Output:

```
{
  "type": "explanation",
  "topic": "zakat",
  "explanation": "LLM-generated explanation grounded in sources...",
  "sources": [
    {"source": "quran", "reference": "Quran 9:60", "content_en": "..."}
  ],
  "processing_time_ms": 2345.6
}
```

Tool 5: furqan_domains

List all available knowledge domains and their statistics.

Input Schema: Empty object {}

Output:

```
{
  "type": "domains",
  "domains": [
    {
      "id": "islamic",
      "name": "Islamic Studies",
      "description": "Quran, Hadith, and Fiqh – verified Islamic scholarly sources.",
      "collections": ["quran", "hadith", "fiqh"],
      "total_sources": 6316
    }
  ]
}
```

6. Intent Detection & Safety

Intent Detection

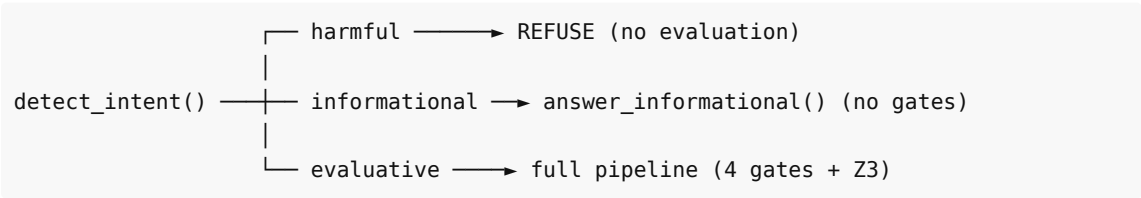
Simple keyword-based classification at the beginning of every evaluation:

```
def detect_intent(question: str) -> str:
    """Returns: 'harmful', 'informational', or 'evaluative'"""
```

Informational signals (prefix matching):

- English: "what is", "who is", "when did", "where is", "define", "explain", "list", "describe"
- Arabic: "ما هو", "ما هي", "من هو", "من هي"

Routing:



Safety Filter — Harmful Keywords

```
_HARMFUL_KEYWORDS = [
    "how to kill", "how to harm", "how to make a bomb",
    "how to poison", "suicide method", "how to attack",
```

```
    "terrorism", "how to destroy",  
  ]
```

When detected, the evaluation is **immediately refused** with no processing.

7. Installation Guide

For OpenClaw

Add to your OpenClaw MCP configuration:

```
{  
  "mcpServers": {  
    "furqan-reasoning": {  
      "command": "python",  
      "args": ["-m", "furqan_raas.mcp_server"],  
      "cwd": "/path/to/al-furqan/furqan-raas"  
    }  
  }  
}
```

For Claude Code

Add to your MCP settings:

```
{  
  "mcpServers": {  
    "furqan-reasoning": {  
      "command": "python",  
      "args": ["-m", "furqan_raas.mcp_server"],  
      "cwd": "/path/to/al-furqan/furqan-raas"  
    }  
  }  
}
```

For Cursor

Add to your Cursor MCP configuration:

```
{  
  "mcpServers": {  
    "furqan-reasoning": {  
      "command": "python",  
      "args": ["-m", "furqan_raas.mcp_server"],  
      "cwd": "/path/to/al-furqan/furqan-raas"  
    }  
  }  
}
```


Manual CLI

```
cd /path/to/al-furqan/furqan-raas
python -m furqan_raas.mcp_server
```

Then send JSON-RPC 2.0 on stdin.

Environment Variables

Variable	Default	Description
(none)	—	RaaS uses engine defaults; configure LLM via al_furqan config

8. SKILL.md Reference

```
---
name: furqan-reasoning
description: Axiom-anchored reasoning engine with formal Z3 verification.
             Evaluates claims against verified sources with deterministic scoring.
---
```

Available commands:

- furqan_evaluate — Full 4-gate evaluation with optional Z3
- furqan_verify — Quick claim verification with confidence scoring
- furqan_retrieve — Knowledge base search
- furqan_explain — Sourced explanation
- furqan_domains — List available domains

Transport: JSON-RPC 2.0 over stdio

Safety: Harmful auto-refused, informational shortcutted, evaluative → full pipeline

9. Example Calls & Responses

Example 1: Evaluative Question

Request:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "furqan_evaluate",
    "arguments": {
      "question": "Is fractional reserve banking ethical?",
      "depth": "standard"
    }
  }
}
```

Response (summary):

```
{
  "type": "evaluation",
  "response": "Fractional reserve banking fails all four survival gates...",
  "verdict": {
```

```

    "gate_scores": [
      {"name": "Source-Integrity", "score": 40, "result": "Fail"},
      {"name": "Structural-Consistency", "score": 30, "result": "Fail"},
      {"name": "Mediation-Zeroing", "score": 20, "result": "Fail"},
      {"name": "Origin-Aware", "score": 0, "result": "Fail"}
    ],
    "total_score": 22
  }
}

```

Example 2: Informational Question

Request:

```

{"jsonrpc": "2.0", "id": 2, "method": "tools/call", "params":
{"name": "furqan_evaluate", "arguments": {"question": "What is zakat?"}}}

```

Response:

```

{
  "type": "informational",
  "response": "Zakat is one of the Five Pillars of Islam...",
  "category": "religion",
  "sources_suggested": ["Quran 9:60", "Bukhari #1395"]
}

```

Example 3: Knowledge Retrieval

Request:

```

{"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params":
{"name": "furqan_retrieve", "arguments": {"query": "الربا", "sources":
["quran"], "limit": 3}}}

```

Response:

```

{
  "type": "retrieval",
  "query": "الربا",
  "results": [
    {"source": "quran", "reference": "Quran 2:275", "content_ar": "...",
"content_en": "..."}
  ],
  "total_found": 3
}

```

10. Test Coverage

File: furqan-raas/tests/test_mcp_server.py — **31 tests**

Category	Tests	Coverage
Protocol	Initialize, tools/list, ping	JSON-RPC 2.0 compliance
Evaluation	Full eval, depths, informational, harmful	All intent paths
Verification	Claim verify, confidence scoring	KB integration
Retrieval	Query, source filtering, limits	Collection search
Explain	Topic explanation, source grounding	LLM + KB
Domains	List domains	Static data
Error handling	Missing params, unknown tools	Edge cases

Furqan RaaS Documentation — March 21, 2026

Variance R&D — The Criterion Project